

Batching, Scheduling, and LLM Serving Systems

pig7selene

September 5, 2026

Abstract

An LLM serving system must turn an irregular stream of autoregressive requests into efficient accelerator work while respecting interactive latency and memory limits. This chapter explains why fixed request-level batches waste capacity, develops static, dynamic, and continuous batching, and distinguishes them from microbatching. It then treats Prefill and Decode as competing workloads, derives a KV-Cache-aware admission condition, and examines queueing, fairness, priority, and preemption policies. The result is a systems model for reasoning about throughput, TTFT, TPOT, and resource reclamation without prescribing a specific serving framework.

1 Introduction

Chapter 19 defined Prefill, Decode, and the latency quantities used to describe one inference request. Chapters 20 and 21 showed that KV Cache state and model representation constrain how many such requests can coexist. A serving system must make those constraints operational. Requests arrive at arbitrary times, contain different prompt lengths, generate an unknown number of output tokens, and occupy different amounts of cache memory as they progress. The accelerator, however, is most efficient when it receives a sufficiently large, regular batch of compatible work.

This mismatch distinguishes LLM serving from a single fixed-shape forward pass. A request cannot usually be dispatched once and forgotten until completion. Each Decode step is another scheduling opportunity, but using that opportunity well requires a scheduler to manage queues, cache capacity, request state, and service objectives together. The chapter studies those decisions. It does not introduce a particular framework, a distributed cluster topology, or Speculative Decoding; those are separate layers built on the scheduling model developed here.

For request i , let a_i denote its arrival time, P_i its prompt length in tokens, and $G_{i(t)}$ the number of tokens generated by time t . Its cached sequence length is

$$S_{i(t)} = P_i + G_{i(t)}. \quad (1)$$

The eventual output length is generally unknown when the request arrives. This uncertainty matters both for queueing and for memory reservation. The notation uses the model dimensions of Chapter 20: L Transformer blocks, g key-value heads, d_h head dimension, and b bytes per cached scalar.

2 Why Request-Level Batching Fails

Static batching forms a batch once, starts the requests together, and retains that membership until the batch reaches a completion boundary. This works well for workloads in which every example has similar known work, such as a fixed-shape classifier. It works poorly for autoregressive generation. A request with a short answer finishes while a longer neighbor continues to Decode; its slot may sit idle, and a new request cannot use it until the original batch has been released. A long prompt can also delay the first token of unrelated short prompts if they must enter the same execution unit.

Dynamic batching improves the boundary between batches. The server collects arrivals for a short waiting interval, forms a batch from the queue, executes it, then forms another batch later. It can raise utilization relative to one-request-at-a-time execution and can limit the delay spent waiting for an

entire fixed batch to fill. However, once a dynamic batch has begun generation, its membership is still normally fixed. It therefore retains the central length-variability problem: requests finish at different iterations while new arrivals wait outside the active set.

The difference becomes acute for a batch with one request that produces ten tokens and another that produces one thousand. At the tenth Decode iteration, the first request has no useful next-token work left. A fixed batch either carries an inactive slot, compacts only at a coarse boundary, or delays a waiting request that could use the released capacity. ORCA formalized **iteration-level scheduling**: the server schedules one generation iteration at a time rather than committing to a request-level batch until all its members finish [1].

2.1 Continuous Batching and Microbatching

Continuous batching applies that iteration-level view throughout serving. After a Decode step, the scheduler may remove completed requests, admit waiting requests whose Prefill work can begin, and regroup the requests eligible for the next Decode step. A request thus enters and leaves the active set independently of its neighbors. The batch remains a useful accelerator unit, but it is no longer a long-lived ownership boundary.

Microbatching is related but different. A microbatch is a smaller execution unit selected from the work that a scheduler has already chosen to run. It may help bound a Prefill chunk, fit a temporary workspace, or create an execution granularity for Pipeline Parallelism. It does not by itself imply that membership can change after every Decode iteration. Continuous batching is a policy over time; microbatching is a way to subdivide selected work for execution.

Mechanism	When membership changes	Main benefit	Main limitation
Static batching	At a full-batch boundary	Simple regular execution	Completed requests and new arrivals cannot promptly exchange slots
Dynamic batching	When a newly formed batch is dispatched	Better arrival aggregation	Membership stays fixed during a generation
Continuous batching	At Decode-iteration boundaries	Reclaims slots and admits work as requests complete	Requires stateful cache and queue management
Microbatching	Within already selected work	Controls execution granularity and temporary resource use	Does not alone solve request admission or length variability

Table 1: Batching names a grouping decision, but the relevant boundary differs. Continuous batching changes request membership during generation; microbatching changes the size of a dispatched execution unit.

The term **continuous** should not imply zero queueing delay or unbounded admission. An arriving request still waits when the active batch, available cache blocks, or latency policy leave no safe place to run it. The essential change is that a completed request’s resources become candidates for reuse at the next iteration, rather than only when an entire historical batch terminates. Modern paged-cache systems make this fine-grained lifecycle practical by allocating and reclaiming cache storage in blocks rather than as fixed maximum-length reservations [2].

3 Workload Phases and Queueing

Prefill and Decode have different scheduling shapes. Prefill processes many known prompt tokens and constructs their cache state. Its cost grows with prompt length and benefits from substantial matrix

computation. Decode processes one new token per active request at each iteration. It is sequential across output positions and commonly has a stronger dependence on reading model weights and KV Cache state. Chapter 19 develops this execution distinction; here it becomes a source of queue contention.

When the phases share the same accelerators, a scheduler must decide whether a large new Prefill should run ahead of the next Decode step for already streaming requests. Giving Prefill too much priority can improve TTFT for the new request but enlarge TPOT for every active Decode request. Giving Decode strict priority preserves streaming cadence but lets a long queue of prompts accumulate and degrades TTFT. DistServe identifies this prefill-decode interference as a reason to consider phase-specific resource allocation in some serving architectures [3]. Co-location remains a valid design when its trade-offs match the workload; phase disaggregation is not a universal requirement.

Head-of-line blocking occurs when work at the front of a queue delays other work that could otherwise make useful progress. A very long Prefill submitted under strict FIFO can delay many short interactive prompts. A long generation can hold cache capacity and slow admission even if later requests need little compute. A scheduler reduces such blocking only by adopting a different policy, for example chunking Prefill work, separating queues, or allowing an eligible short request to run first. Each remedy changes fairness or overhead; no policy removes the underlying variability for free.

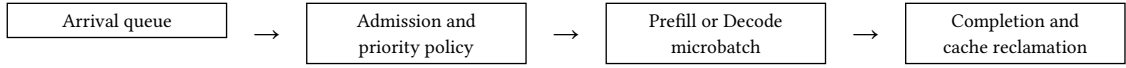


Figure 1: At an iteration boundary, a serving scheduler decides which queued or active requests run next. Completion and cache reclamation feed capacity back into the next decision.

The queue in Figure 1 should represent work state, not merely request IDs. It commonly records arrival time, prompt length, current generated length, stop condition, priority class, predicted or declared output limit, cache-block ownership, and whether the request is awaiting Prefill, active Decode, swapped state, or completion. These fields make policy decisions auditable and permit a cancellation or completion event to release exactly the state it owns.

4 Memory-Aware Admission and Reclamation

A candidate request can be computationally cheap enough to run and still be impossible to admit. Its initial Prefill requires cache capacity for P_i tokens, and every later Decode iteration grows its state. Ignoring block rounding, prefix sharing, and metadata, its own cache payload has the Chapter 20 form

$$m_{i(S_i)} = 2Lgd_h bS_i. \quad (2)$$

At scheduling epoch n , let A_n be the proposed active set, M_{weights} the resident model footprint, and $M_{\text{workspace}}$ the temporary memory needed by selected kernels. A conservative one-step feasibility condition is

$$M_{\text{weights}} + \sum_{i \in A_n} m_{i(S_i+1)} + M_{\text{workspace}} \leq M_{\text{device}}. \quad (3)$$

Equation 3 does not reserve every request’s maximum possible answer. It asks whether the next selected step fits after allowing each active Decode request to append one token. A server may instead reserve a declared output limit, use a statistical length estimate, or admit optimistically and preempt later under pressure. Those alternatives trade utilization against the chance that a live request will later need eviction, swapping, or recomputation.

Admission control is the rule that accepts, defers, sheds, or routes a request before it consumes scarce execution and cache resources. Under high load, accepting every arrival can make all requests slow: queue delays grow, KV Cache capacity becomes fragmented or exhausted, and a late out-of-memory failure wastes earlier work. Deferral keeps the request queued for a future iteration; rejection returns a capacity result promptly; routing selects another compatible replica or service tier. The correct choice depends on the service contract, not only on the local accelerator’s instantaneous utilization.

Request completion is the dual operation. A stopping condition, cancellation, error, or timeout releases its private cache blocks and removes its Decode work from the next active set. Shared prefix blocks need reference-aware reclamation, as Chapter 20 explains. Reclamation should occur before the next admission decision so the scheduler can reuse capacity immediately. Delayed or incomplete cleanup produces an apparent memory leak and artificially lowers batch occupancy even when model compute is idle.

4.1 Preemption Under Resource Pressure

Preemption temporarily removes a live request from active execution so another request can run. Autoregressive generation permits a natural boundary between tokens: preempting after a Decode iteration preserves a coherent cache prefix. It does not make preemption costless. A server has three broad choices for the interrupted state. It may retain the KV Cache in device memory and simply stop issuing Decode work; it may swap cache state to another memory tier and later restore it; or it may discard the state and recompute Prefill when the request resumes.

Retaining state has the lowest resume cost but does not free the resource that caused pressure. Swapping can free accelerator memory but introduces transfer delay and bandwidth contention. Recomputing avoids transfer storage but repeats model work proportional to the cached prefix. FastServe explores token-granularity preemption and cache swapping as ways to reduce head-of-line blocking under variable request lengths [4]. The practical policy must account for the request’s cache size, expected wait, priority, and the device-to-host transfer path rather than treating all paused requests alike.

5 Scheduling Policies and Service Objectives

FIFO serves eligible requests by arrival order. It is easy to explain, makes queue order visible, and can provide a strong fairness baseline. Its weakness is that it does not distinguish a short request from a long one, so a costly front request can create head-of-line blocking. A shortest-job-oriented heuristic tries to favor smaller prompt lengths, output limits, or predicted remaining work. It can improve mean latency when estimates are useful, but generated length is uncertain and an aggressive short-first policy can starve long requests.

Priority scheduling gives selected classes precedence, such as an interactive user over an offline batch job. It is appropriate only when the service contract declares that distinction. Fairness-aware scheduling limits how much service a tenant, request, or priority class can receive over an interval. Aging, virtual service counters, or quotas can prevent a stream of high-priority or short requests from indefinitely delaying another request. These policies are not interchangeable: FIFO emphasizes order, shortest-job heuristics emphasize mean completion time, priority emphasizes declared importance, and fairness emphasizes bounded inequity.

Policy	Primary signal	Potential benefit	Principal risk
FIFO	Arrival time	Transparent baseline and low scheduling overhead	Long requests cause head-of-line blocking
Shortest-job heuristic	Prompt size, output cap, or length prediction	Can reduce mean waiting time	Prediction error and starvation of long work
Priority scheduling	Declared service class or deadline	Protects latency-sensitive work	Low-priority requests can become unbounded
Fairness-aware scheduling	Accumulated service or tenant share	Bounds inequity across request classes	May sacrifice peak local throughput

Table 2: A scheduling policy selects which objective is protected when requests compete. The policy should be stated with its service scope, not inferred from a single throughput number.

Policies also decide how to interleave Prefill and Decode. A system can reserve part of each iteration for new prompts, alternate phase-focused microbatches, cap the number of concurrent Prefills, or use separate queues with an explicit budget. The chosen rule should expose its intent: a maximum queueing delay, a TPOT target, a minimum batch occupancy, or a cost objective. A scheduler that silently optimizes only aggregate tokens per second may deliver excellent offline throughput while producing unacceptable interactive waiting time.

6 Metrics and Throughput–Latency Trade-offs

For a request whose first emitted token arrives at t_i^{first} and whose final token arrives at t_i^{last} , the first-token latency is

$$\text{TTFT}_i = t_i^{\text{first}} - a_i. \quad (4)$$

If it emits $G_i^{\text{final}} > 1$ output tokens, a representative average inter-token quantity is

$$\text{TPOT}_i = \frac{t_i^{\text{last}} - t_i^{\text{first}}}{G_i^{\text{final}} - 1}. \quad (5)$$

Queueing delay is the part of TTFT spent awaiting first execution rather than computing Prefill. A system should record it separately, because a high TTFT can arise from overload, a policy choice, or slow prompt computation. Aggregate output throughput can be reported as completed output tokens per second; request throughput as completed requests per second. Neither identifies tail latency, fairness, or how much work was rejected under pressure.

Batch occupancy describes how much of the selected execution capacity is populated by useful request work. GPU utilization measures whether accelerator resources are busy, but a high value can still be spent on poorly prioritized long Prefills or on recomputation after preemption. KV Cache utilization measures used cache payload or allocated blocks relative to cache capacity. It should be reported with fragmentation, reservation, and eviction behavior so that a full allocator is not mistaken for a healthy serving workload.

The central tension is therefore not one-dimensional. Larger batches can improve matrix-kernel efficiency and total throughput, but waiting to form them raises queueing delay. Prefill-heavy batches can improve prompt-processing efficiency, but may disrupt active streaming requests. Strict Decode priority can improve TPOT, but can make TTFT worse. A responsible evaluation reports TTFT and TPOT distributions, queue delay, throughput, batch occupancy, and KV Cache utilization under a declared arrival mixture and service-level objective, rather than choosing one favorable aggregate number.

7 Implementation Contracts

The request contract must specify tokenizer and model revision, prompt token sequence, maximum generated length, stopping rule, arrival timestamp, service class, cancellation semantics, and any declared latency objective. The scheduler state must distinguish queued, prefill-active, decode-active, paused, swapped, completed, failed, and cancelled requests. A request must never occupy two incompatible states, and a completion event must be idempotent so that duplicate cancellation signals do not double-free cache blocks.

The memory contract connects scheduler accounting to Chapter 20’s cache representation. For every candidate action, the implementation should account separately for model weights, active cache payload, reserved or rounded cache blocks, shared-prefix references, temporary workspaces, and swap buffers. Admission, growth, preemption, and reclamation must be atomic with respect to the allocator: a request cannot be selected for the next Decode step if the blocks needed for that step have already been reassigned. Tests should cover a batch boundary, simultaneous completion and admission, cancellation during queueing, a block-growth boundary, and memory pressure that forces every configured preemption path.

The metric contract records arrival, first-token, each-token, final-token, queueing, preemption, and reclaim timestamps under one clock. It should expose policy name and parameters, batch composition by phase, rejected or deferred requests, cache utilization, and kernel path. Evaluation must compare policies under the same prompt distribution, output-length limits, hardware, model representation, cache policy, and service objective. Otherwise an apparent scheduler improvement may merely be a difference in workload, quantization, or memory budget.

8 Summary

LLM serving turns a variable stream of sequential generation requests into accelerator work. Static and ordinary dynamic batching lose capacity because their membership remains fixed while output lengths diverge. Continuous batching instead reschedules at Decode-iteration boundaries, removes completed requests, and admits new eligible work; microbatching is a separate execution-granularity tool.

The scheduler must balance Prefill and Decode, TTFT and TPOT, throughput and fairness, and compute admission against KV Cache capacity. Its decisions are constrained by the persistent memory of each request, not only by its next forward-pass cost. Admission control, resource reclamation, and preemption make those constraints explicit. The next chapter can therefore treat Speculative Decoding as a change to the work performed per generation step, while this chapter supplies the system that decides which requests may take that step.

References

- [1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “ORCA: A Distributed Serving System for Transformer-Based Generative Models,” in *16th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/yu>
- [2] W. Kwon *et al.*, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, Association for Computing Machinery, 2023, pp. 611–626. doi: 10.1145/3600006.3613165.
- [3] Y. Zhong *et al.*, “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2024, pp. 193–210. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
- [4] B. Wu *et al.*, “FastServe: Fast Distributed Inference Serving for Large Language Models,” *arXiv preprint arXiv:2305.05920*, 2023, doi: 10.48550/arXiv.2305.05920.